جزوهٔ درسی · جلسهٔ ۵ از ۵

توسعهٔ مبتنی بر مشخصات در سطح تولید

از نمونهٔ شهودی تا واقعیت تولید: مشخصات، فرهنگ تیم، و شبکهٔ ایمنی اعتماد صفر

مطالعهٔ پیشنهادی: ۵۰ دقیقه **شامل:** جمع‌بندی کل دوره

فهرست مطالب

1. مقدمه: توهم سرعت
2. توسعهٔ مبتنی بر مشخصات
3. مشخصات خوب چه شکلی است
4. قالب و اقتصاد توکن
5. توسعهٔ رفتارمحور
6. دستورها کجا نگه‌داری می‌شوند
7. پنج حالت اجرا
8. فرهنگ تیم و تحول فرایند
9. سه لایهٔ بازبین خودکار
10. پایداری و خستگی تأیید
11. توسعهٔ اعتماد صفر
12. جمع‌بندی دوره
13. خودآزمایی

۱. مقدمه: توهم سرعت

> «کدنویسی شهودی (Vibe coding)» به‌معنای «شهودی در تولید» نیست.

روال روزانهٔ یک مهندس نرم‌افزار در یک سال گذشته کاملاً وارونه شده است. پیش‌تر زمان زیادی صرف کندوکاو در مستندات و رابط‌ها می‌شد، و پس از پایان کد، تلاش چشمگیری برای دیباگ (debug) و رفع فاصلهٔ میان کد کارکردی

و قصد (intent) اولیه لازم بود. امروز توسعه با سرعت سرسام‌آوری پیش می‌رود: تیم‌ها از عامل‌های (agent) کدنویسی استفاده می‌کنند که فقط متن پیشنهاد نمی‌دهند، بلکه واقعاً ابزار (tool) به کار می‌گیرند و کار اجرا می‌کنند. حس می‌کنید لشکری از کارآموز استخدام کرده‌اید که نه می‌خوابند و نه شکایت می‌کنند.

> **اما یک نکته هست**
>
> در حالی که سرعت به اوج رسیده، **توهم سرعت** (Illusion of Speed) واقعی است. نسبت اشکال به کد همچنان چالش است: چون هوش مصنوعی کد را بسیار سریع‌تر می‌نویسد، می‌تواند خطاهای بالقوه را هم با نرخی بی‌سابقه تولید کند. وقتی عامل دچار توهم (hallucination) می‌شود — یعنی مدل با اطمینان چیزی از خود می‌سازد که درست نیست — فقط یک اشکال نمی‌سازد؛ می‌تواند هزار خط منطقِ «هم‌حس اما از نظر کارکرد معیوب» تولید کند.

و اگر بازبینان انسانی زیر سیل پول‌ریکوئست‌های (pull request) تولیدشده غرق شوند، سرعت نوشتن بی‌ربط می‌شود. فرایند واقعاً سریع‌تر نشده؛ فقط انبوه بزرگ‌تری برای مرتب‌کردن بعدی ساخته‌اید.

> **جابه‌جایی گلوگاه — و فرصت پنهانش**
>
> چون پیاده‌سازی دیگر گلوگاه اصلی نیست، می‌توان هوش مصنوعی را به نوشتن **پوشش** (coverage) **تستی جامع‌تر از آنچه هر انسانی در همان بازه می‌توانست بنویسد** گماشت. این راهی نیرومند و برنامه‌پذیر برای بالابردن اطمینان به کد است. افزون بر این، توان هوش مصنوعی بسیار فراتر از پیاده‌سازی می‌رود: در طراحی سامانه، نوشتن مشخصات، برنامه‌ریزی نقشهٔ راه و تحلیل نتایج هم عالی است.

در محیط سازمانی، به این «حس‌وحال» می‌گوییم **توسعه با هوش مصنوعی عامل‌محور**. برخلاف هوش مصنوعی مولد استاندارد که مثل تکمیل خودکار هوشمند عمل می‌کند، هوش مصنوعی عامل‌محور مانند **عضوی ترکیبی از تیم** است: از مدل زبانی برای تولید استفاده می‌کند (مغز) و از ابزارها برای یکپارچه‌شدن در گردش‌کار (workflow) (دست‌ها). می‌تواند استدلال کند، مشخصات (spec) یا پروندهٔ تست (test) بنویسد، با مرورگر رابط را بیازماید، و کد را ثبت و ادغام کند. اما وقتی توسعه از آزمایش فردی به تولید تیمی در مقیاس بزرگ می‌رسد، روشن می‌شود که **نوشتن هزار خط کد پیش از ناهار، همان انتشار نرم‌افزار نیست.**

۲. توسعهٔ مبتنی بر مشخصات

در دنیای سنتی مهندسی نرم‌افزار، به توسعه‌دهندگان آموخته بودند «کد-اول» باشند: ایده‌ای مبهم منجر به بازکردن ویرایشگر و تایپ‌کردن می‌شد تا چیزی کار کند. اما در عصر هوش مصنوعی عامل‌محور، فعالیت روزانه به‌شدت جابه‌جا شده است. بیشتر وقت حالا صرف نوشتن **مشخصات باکیفیت** می‌شود: دستورهای فنی دقیقی که به هوش مصنوعی می‌گویند دقیقاً چه بسازد. در این گردش‌کار، نقش توسعه‌دهنده بیشتر **معمار فنی** است تا کدنویس سنتی.

بخش حیاتی ماجرا: کد دورریختنی (Disposable code) است

اگر مشخصاتی محکم نوشته شود، کل کدبیس بارها و بارها بازتولیدشدنی است. حتی می‌توان به عامل گفت کل پروژه را در یک بعدازظهر از یک زبان به زبان دیگر ببرد.

و چون کد دورریختنی است، آن دلبستگی عاطفی قدیمی وجود ندارد. وقتی دوازده ساعت صرف دیباگ یک نقطه‌ویرگول نکرده‌اید، از دورانداختن و شروع دوباره — وقتی نیازمندی‌ها عوض می‌شوند — هم نمی‌ترسید.

عامل‌های کدنویسی با استفاده از مدل زبانی به‌عنوان «مغز» برای استدلال و ابزارها به‌عنوان «دست‌ها» برای اجرا کار می‌کنند. **اگر به مغزی «حس‌وحال» بدهید به‌جای «نقشه»، حدس می‌زند.** و در نرم‌افزار سازمانی، حدس‌زدن همان چیزی است که حادثهٔ «عامل یاغی (Rogue agent)» را می‌سازد.

۳. مشخصات خوب چه شکلی است

مشخصات خوب یک **ستارهٔ شمالی معماری** (Architectural North Star) است. از «پاره‌پاره‌شدن زمینه (Context fragmentation)» جلوگیری می‌کند — معادل دیجیتالی بازی «تلفن»، جایی که هوش مصنوعی رشتهٔ ماجرا را گم می‌کند چون دارد به عکس‌های قدیمی فایل‌ها نگاه می‌کند. می‌توان از هوش مصنوعی برای پالایش مشخصات استفاده کرد، عملاً به‌عنوان هم‌نویسنده یا ویراستار و بازبین. معمولاً مشخصات درون خود کدبیس ذخیره می‌شود، مثلاً به‌صورت فایلی مارک‌داون یا YAML در پوشهٔ `specs/`، و **منبع حقیقت** برای هم انسان و هم هوش مصنوعی است.

چهار بخش مشخصاتِ تولید یک پروژهٔ تازه

1. **طراحی فنی کامل:** فقط نگویید «صفحهٔ ورود بساز». تجزیه‌اش کنید: نیازمندی‌ها، اسکیمای پایگاه داده (ساختار داده‌ها) و مشخصات رابط‌ها (قراردادهایی که اجازه می‌دهند بخش‌های مختلف نرم‌افزار با هم حرف بزنند).
2. **کمک بصری:** نمودارها، و فهرستی از ابزارها و کتابخانه‌های مشخص **همراه با شمارهٔ نسخه**.
3. **اطلاعات زمینه‌ای:** «چرا» را پشت «چه» به عامل بدهید؛ این کمک می‌کند رو به جلو فکر کند و گام‌هایی را که احتمالاً لازم خواهید داشت هم بشناسد.
4. **سناریوها:** خوب چه شکلی است، بد چه شکلی است، و حالت‌های مرزی کدام‌اند.

نکتهٔ عملی

طراحی‌های فنی را جایی بنویسید که دیگران بتوانند بخوانند و بازبینی کنند. این حالا مهم‌تر از همیشه است: ترجیح می‌دهید **انسانی** نقص منطقی طرح شما را بگیرد، تا اینکه صبر کنید هوش مصنوعی هزاران خط کد معیوب تولید کند. پس از بازبینی، سند را به مارک‌داون تبدیل و در پوشهٔ `specs/` فضای کاری‌تان بگذارید.

۴. قالب و اقتصاد توکن

پژوهش‌ها نشان می‌دهند عامل‌های مدل‌زبانی به **نحوهٔ قالب‌بندی دستورها** حساسیت شدیدی دارند، تا حدی که فایل‌های مارک‌داون عمومی و بهینه‌نشده می‌توانند افت عملکرد چشمگیری بسازند. راهبرد پیشنهادی، رویکردی **ترکیبی** است: از سرتیترهای تمیز مارک‌داون برای لنگرانداختن توجه مدل استفاده کنید، اما برای هر پیکربندی ساختارمند یا اسکیمای داده‌ای با **عمق تودرتویی بیش از سه**، به YAML سوئیچ کنید.

قالب	دقت تجزیه (decomposition) در ساختارهای عمیقاً تودرتو
YAML	بالاترین دقت
JSON	متوسط
XML	پایین‌ترین

با رندرکردن مشخصات عمیقاً تودرتو در YAML و نگه‌داشتن دستورهای روایی در مارک‌داون، **هزینهٔ قالب** (Format tax) را دور می‌زنید؛ همان هزینهٔ استدلالی که ورودی‌های سنگین تحمیل می‌کنند.

توکن (token) یک قید (constraint) فیزیکی است

مدل‌های زبانی ساختار داده را تفسیر نمی‌کنند؛ متنِ نشانه‌گذاری‌شده را پردازش می‌کنند. هر نویسه‌ای که می‌فرستید به نشانه‌ها شکسته می‌شود و هر نشانه بودجه، زمان و ظرفیت زمینه (context) مصرف می‌کند. هر فاصلهٔ غیرضروری در یک بلوک عمیقاً تودرتو و هر دستور (instruction) تکراری، چرخهٔ پردازش و ظرفیت توجه را در همان حلقه‌های (loop) چندنوبتی می‌بلعد که عامل در آن‌ها برنامه را می‌سازد، می‌آزماید و ویرایش می‌کند.

پس با پوشهٔ `specs/` نه مثل مستندات، بلکه مثل **یک مجموعه‌دستور کامپایل‌شده و کم‌چرب** رفتار کنید.

۵. توسعهٔ رفتارمحور

مشخصات رفتارمحور، ابزار نهایی تبدیل ایده‌های مبهم انسانی به طراحی معماری دقیقی است که عامل بتواند بدون حدس‌زدن بسازدش. در هسته‌اش، روشی است که با زبان طبیعیِ ساده و ساختارمند توصیف می‌کند سامانه **از دید کاربر** دقیقاً چگونه باید رفتار کند — پیش از آنکه هیچ کدی نوشته شود.

این مشخصات از نحوی استاندارد به نام **گرکین** استفاده می‌کند که بر قالبی ساده و اعلانی تکیه دارد: **سناریو / با فرض / وقتی / آنگاه**. این قالب مدل را وادار می‌کند به‌صورت **موقعیت** ← **کنش** ← **نتیجه** فکر کند، و همین کدنویسی شهودی را کاملاً حذف و عامل را روی ریلی سخت‌گیرانه نگه می‌دارد.

```
Scenario: Retry a failed payment within the allowed window
  Given a payment with status "failed" and 2 previous attempts
  And the failure reason is "network_timeout"
  When the retry job runs within 24 hours of the original attempt
  Then a new attempt is created with attempt_number 3
  And the original transaction id is preserved
  And no duplicate charge is issued

Scenario: Refuse retry after the attempt limit
  Given a payment with status "failed" and 3 previous attempts
  When the retry job runs
  Then no new attempt is created
  And the payment is moved to status "requires_manual_review"
```

سه کار با یک نوشته

این متن هم‌زمان **مشخصات** است، هم **پروندهٔ تست**، و هم **دستور دقیق برای عامل**. همین سه‌کاره‌بودن است که توسعهٔ رفتارمحور (BDD) را در عصر عامل‌ها دوباره ارزشمند کرده است.

۶. دستورها کجا نگه‌داری می‌شوند

برای عمل به توسعهٔ مبتنی بر مشخصات، باید فهمید ابزارهای کدنویسی چگونه دستورها را مصرف می‌کنند. دستورها در یک جا نوشته نمی‌شوند. ریختن یک سند طراحی صدصفحه‌ای مستقیم در پنجرهٔ گفت‌وگو، بودجهٔ زمینهٔ کوتاه‌مدت را به‌سرعت تمام می‌کند، تأخیر (latency) را بالا می‌برد و زمینه را پاره‌پاره می‌کند.

جایگاه	ماهیت	چه چیزی آنجا می‌رود
رابط گفت‌وگو	کوتاه‌عمر، ویژهٔ نشست (session)	فقط برای هماهنگی سطح‌بالا و حلقهٔ بازخورد فوری. برای نمونه: «طراحی موجود در `specs/payment_retry.md` را بازبینی کن و تست‌های واحدِ شکست‌خوردهٔ تعریف‌شده در سناریوی ۳ را بساز.»
پوشهٔ مشخصات	ویژهٔ کار، در کنترل نسخه	پوشه‌ای ایستا که مستقیم در ریپو (repository) ثبت می‌شود و طراحی فنی، سناریوهای رفتارمحور، قرارداد APIها و اسکیماهای ساختارمند را نگه می‌دارد. عامل این دایرکتوری را پویا (dynamic) نمایه می‌کند و بدون چپاندن دستی پرامپت، کد را می‌سازد و راستی‌آزمایی می‌کند.
مهارت‌های عامل (skill)	قابل‌استفادهٔ مجدد، رفتارمحور	فایل‌های ساختارمندی که گردش‌کارهای تخصصی و محرک‌دار را حمل می‌کنند و باید در دایرکتوری مشخصی قرار بگیرند تا شناسایی شوند. آن‌ها **عادت‌های مهندسی تکرارشونده** را می‌آموزند — مثل نگه‌داری خودکار فهرست تغییرات وقتی تغییری در کد تشخیص داده می‌شود. این پوشه می‌تواند دارایی داده یا اسکریپت هم داشته باشد.
پرامپت‌های سامانه (prompt)	سراسری، هویت‌محور	جایی که هوش مصنوعی **دی‌ان‌ای مهندسی** شما را می‌آموزد. فایل‌های زمینه سلسله‌مراتبی خوانده و به‌هم چسبانده می‌شوند: **پروفایل سراسری** در دایرکتوری خانگی که شخصیت و سبک پیش‌فرض را تعریف می‌کند؛ **پیکربندی مشترک چندابزاری** که از پاره‌پاره‌شدن دستورها میان چند کلاینت جلوگیری می‌کند؛ و **مشخصات پروژه** در ریشهٔ ریپو که بالاترین اولویت را دارد.

۷. پنج حالت اجرا

فقط یک راه برای تبدیل مشخصات به کد وجود ندارد. هر حالت، ذهنیت متفاوتی می‌خواهد.

۱. تولید پروژه — معمار

- **بدون حالت بی‌محابا** (`YOLO mode`): صریحاً به عامل بگویید فوری کد نزند. اول باید ساختار پوشه‌ها و پشتهٔ فناوری را برای تأیید پیشنهاد بدهد.
- اطمینان حاصل کنید که پرامپت شامل تولید **تست، مستندات و گزارش‌برداری** است — گزارش‌برداری همان «جعبهٔ سیاه» دیجیتالی است که ثبت می‌کند برنامه چه می‌کند.
- برای هر کتابخانه **شمارهٔ نسخه** بدهید. بدون آن، عامل ممکن است نسخهٔ قدیمی‌تری پیشنهاد دهد، چون تاریخ پایان دادهٔ آموزشی‌اش در گذشته بوده است.

> **نکتهٔ نسخه‌ها**
> گاهی عامل نسخه‌های پایین‌تری از ابزارها، کتابخانه‌ها یا مدل‌های موجود را پیشنهاد می‌دهد، چون مدل زیرین تا زمان مشخصی آموزش دیده است. این را می‌توان با افزودن مستندات به‌روز در پوشهٔ مشخصات، یا ذکر این اطلاعات در مهارت‌ها و پرامپت‌های پروفایل پیشگیری کرد. به یاد داشته باشید عامل‌ها به‌آسانی به دادهٔ آموزشی‌شان برمی‌گردند؛ پس شماره‌نسخه‌های پیشنهادی باید همیشه دوباره وارسی شوند.

۲. تولید قابلیت — سازنده

در این حالت، قابلیت‌ها روی کدبیس موجود پیاده می‌شوند. از عامل بخواهید **سبک موجود را رعایت کند**، شامل الگوهای نام‌گذاری و شیوهٔ مدیریت خطا در کد. وقتی چند فایل ویرایش می‌شود، تغییرها باید دستی تأیید شوند. دیدن «تفاوت‌ها» — فهرستی که دقیقاً نشان می‌دهد چه خط‌هایی اضافه یا حذف می‌شود — درون ویرایشگر اهمیت دارد.

۳. رفع اشکال — متخصص جرم‌شناسی

وقتی چیزی خراب شد، حالت جرم‌شناسی لازم است. هدف، تحلیل علت ریشه‌ای و اصلاحی دقیق و هدفمند است. از **پرامپت علامت‌محور** («دکمه کار نمی‌کند») به **پرامپت شاهدمحور** بروید («گزارش‌هایی که با این فرمان گرفتم خطای ۴۰۳ نشان می‌دهند»). سپس جریان را توضیح دهید: «درخواست به متعادل‌کنندهٔ بار می‌رسد ← لایهٔ احراز هویت (`identity`) سرایند را حذف می‌کند ← غلاف شکست می‌خورد.»

- همیشه اول یک **تست واحدِ شکست‌خورده** یا فرمان بازتولید بخواهید تا اشکال را بازسازی کند، و آن پرونده را در کدبیس نگه دارید تا اشکال برنگردد.
- قید سختی بگذارید که عامل **فقط** علت ریشه‌ای را درست کند. کد بی‌ربط نباید «تمیزکاری» شود، چون فرایند بازبینی را پیچیده می‌کند.
- تغییر نام متغیرها پذیرفتنی است، اما باید کاری **جداگانه** باشد، نه بخشی از رفع اشکال.

مرورگر داخلی و تست سرتاسری

برخی محیط‌های عامل‌محور مرورگری داخلی دارند که به عامل اجازه می‌دهد خودگردان (autonomous) برنامهٔ محلی را اجرا کند، با رابط زنده تعامل کند و اصلاح‌های بصری را در زمان واقعی راستی‌آزمایی کند. نکتهٔ امنیتی مهم این است که این زیرعامل (subagent) درون نمایهٔ مرورگری کاملاً ایزوله و سندباکس‌مانند اجرا می‌شود که ورودهای فعال یا نشست‌های شخصی را نمی‌بیند — عملاً محیطی ناشناس و تمیز — و همین از نشت (leak) زمینه و عملیات تصادفی روی حساب واقعی جلوگیری می‌کند. پس پرامپت‌های دیباگ می‌توانند بسیار فراتر از تحلیل ایستای کد بروند.

۴. نوشتن مستندات — نویسنده

در توسعهٔ مبتنی بر مشخصات، **مستندات منبع حقیقت است.** اگر مستندات و کد هم‌گام نباشند، هوش مصنوعی شروع به توهم می‌کند. در مهارت‌های عامل مشخص کنید که دستورهای فایل راهنما و فهرست تغییرات همیشه باید نگه‌داری شوند. از قالب‌های استاندارد مستندسازی درون‌کد استفاده کنید؛ این‌ها روش‌های ساختارمند نوشتن توضیحات‌اند که به هوش مصنوعی (و انسان) کمک می‌کنند بفهمند یک تابع دقیقاً چه می‌کند، بدون خواندن هر خط منطق.

۵. مهندسی داده — کتابدار

هنگام پرس‌وجوی جدول‌ها یا جابه‌جایی فایل‌ها، نقش شما مهندس داده است. از افزونه‌های ویرایشگر برای دسترسی مستقیم به دادهٔ ابری استفاده کنید، و از عامل بخواهید **همیشه پرس‌وجو یا فرمان دقیقی را که برای تولید خروجی به کار برده نشان بدهد.**

۸. فرهنگ تیم و تحول فرایند

کار با عامل‌های کدنویسی مدرن نیازمند تغییر در تفکر و فرهنگ تیم است. چسبیدن به فرایندهای قدیمی هنگام استفاده از ابزارهای مدرن، مثل نصب موتور جت روی درشکه است؛ این فناوری را نمی‌توان به گردش‌کاری بیست‌ساله پیچ کرد و انتظار پرواز داشت.

سه دستهٔ خطای فرایندی

- **تعارض‌های ادغام:** چند توسعه‌دهنده ظرف یک ساعت سراغ یک فایل می‌روند.
- **گره‌خوردگی بازبینی** (Review gridlock)**:** پول‌ریکوئستی غول‌آسا به عروسک روسیِ پول‌ریکوئست‌های تودرتو بدل می‌شود. اولی بدون دومی ادغام نمی‌شود، دومی به سومی نیاز دارد، و سومی پشت بازبینی در منطقهٔ زمانی

دیگری گیر کرده است. برخی تغییرها تأیید می‌شوند در حالی که تغییرهای مرتبط منتظر بازبینی‌اند، و همین تعارض‌های بیشتری می‌زاید.

- **پاره‌پاره‌شدن زمینه:** در غیاب یک توسعه‌دهنده، هم‌تیمی متغیری را در فایل مشترک تغییر نام می‌دهد؛ عامل که به عکسی قدیمی استناد می‌کند، کدی می‌سازد که تابعی دیگر موجود را صدا می‌زند.

پنج راهبرد یکپارچگی پرسرعت

1. **خلاصه و ارزیابی** (evaluation) **مخاطرهٔ بسته‌بندی‌شده:** هر پول‌ریکوئست باید نمای تولیدشده‌ای از آنچه تغییر کرده، نقاط محتمل بروز خطا و ارزیابی مخاطره داشته باشد — به‌صورت فایلی مارک‌داون یا توضیح مفصل ثبت. این کمک می‌کند بازبین انسانی به‌جای گم‌شدن میان خط‌ها، روی اثر معماری تمرکز کند.
2. **بازتعریف مالکیت:** تمرکز بازبینی انسانی را از ایرادگیری سبکی روی کد دورریختنیِ عامل‌نوشته، به تضمین یکپارچگی نقشه‌های معماری منتقل کنید. دغدغه‌های سبکی با ابزارهای خودکار یکپارچه‌سازی مثل لینترهای مشترک یا کتاب‌سبک‌های ویژهٔ فضای کاری حل می‌شوند.
3. **تأیید مشروط** (Conditional LGTM)**:** برای حذف تأخیرهای دوازده‌ساعته در تیم‌های چندمنطقه‌ای، بازبین پول‌ریکوئست را مشروط به پاس‌شدن همهٔ تست‌های خودکار تأیید می‌کند؛ اگر تست‌ها سبز شدند، کد خودکار ادغام می‌شود.
4. **تعریف فرهنگ بدون سرزنش:** در محیط‌های پرسرعت، کسی که بیشترین کد را تولید می‌کند اغلب قربانی سرزنش اشکال‌ها یا تعارض‌های ادغام می‌شود. مفید است دربارهٔ نسبت‌دادن این مسائل به **فرایند یکپارچگی معیوب** به‌جای فردِ استفاده‌کننده از عامل گفت‌وگو کنید.
5. **بازبینی کد به‌دست عامل:** می‌توانید مهارتی بسازید که بازبینی را برایتان انجام دهد، و حتی مهارتی که به بازبینی‌ها پاسخ بدهد. این را می‌توان با گردش‌کارهای خودکار ریپو به‌طور کامل خودکار کرد.

> **و یک پرسش باز برای تیم**
>
> اگر می‌توانید با جوخه‌ای از عامل‌ها کار کنید، آیا اصلاً لازم است به‌صورت تیمی کار کنید؟ اگر پاسخ مثبت است، راه‌هایی برای تقسیم کار بیابید که اعضای تیم به‌ندرت به فایل‌های یکسان دست بزنند — مثلاً مالکیت روشن بر رابط‌ها در برابر تجربهٔ کاربری. وقتی هم‌پوشانی ناگزیر است، «مالک بخش» تعیین‌شده هماهنگی نهایی را انجام دهد.

نمونهٔ مهارت بازبینی کد

```
Act as a Senior Software Engineer and Security Researcher.
Review the provided PR or diff using these strict criteria:

1. Critical Vulnerabilities: hardcoded secrets (API keys),
   SQL injection, XSS, or broken authentication.
2. Logic & Efficiency: off-by-one errors, infinite loops,
   redundant API calls.
3. Readability: better naming conventions, break down
   mega-functions into smaller pieces.
4. Edge Cases: What happens if the input is null?
   What if the network fails?

Output Format:
- Description: what is this PR doing? Explain in detail.

ISSUES:
- Critical:       (stop-ship issues)
- Warnings:       (code smells or style issues)
- Best Practices: (specific lines to refactor for performance)
- Quick Win:      (one-sentence summary of the biggest improvement)

When there are no issues, return the description and "LGTM".
```

۹. سه لایهٔ بازبین خودکار

می‌توانید پرامپت بازبینی عالی بنویسید. اما چه کسی آن را روی هر پول‌ریکوئست و هر پویش شبانه اجرا می‌کند، بدون اینکه انسانی دکمه را فشار دهد؟ گام بعدی، دیپلوی `(deploy)` این مهارت‌ها به‌عنوان **عامل‌های تحلیل پیوستهٔ کد** است: سرویس‌هایی که ریپو را می‌پایند، به رویدادها واکنش نشان می‌دهند و یافته‌ها را بدون درخواست کسی پست می‌کنند. آن‌ها چیزهایی را می‌گیرند که انسان خستهٔ عصر جمعه از دست می‌دهد: وابستگی‌ای با آسیب‌پذیری تازه‌کشف‌شده، یا یادداشتی از شش ماه پیش که بی‌سروصدا به شکاف امنیتی بدل شده است.

لایه	چیست	موازنه
۱ — مدیریت‌شده	مسیر `(trajectory)` آماده: سرویسی را روی سازمان ریپو فعال می‌کنید و هر پول‌ریکوئست بازبینی می‌شود که دربارهٔ سبک، اشکال و امنیت نظر می‌دهد.	هیچ پرامپتی نمی‌نویسید و هیچ زیرساختی مدیریت نمی‌کنید. اما نظرهای بازبینیِ **ارائه‌دهنده** را می‌گیرید نه نظرهای خودتان، و بازبین عمومی چیزی را که برای تیمی با سبک خانگی سخت‌گیرانه یا نمای مخاطرهٔ حوزه‌ای اهمیت دارد از دست می‌دهد.
۲ — ترکیبی	مهارت بازبینی خودتان را می‌نویسید، در ریپو ثبت می‌کنید، و آن را از کنشی در سی‌آی `(CI)` اجرا می‌کنید که خط فرمان عامل را در حالت غیرتعاملی می‌راند و نتیجه را به‌عنوان نظر پست می‌کند.	**مسیر میانی و نقطهٔ شروع درست برای بیشتر تیم‌ها.** رانتایم `(runtime)` مال ارائه‌دهندهٔ یکپارچه‌سازی شماست؛ اما پرامپت‌ها، انتخاب مدل، سندباکس `(sandbox)` و معیارهای بازبینی مال شماست.
۳ — سفارشی	بازبین را به‌عنوان عاملی کامل می‌سازید، روی رانتایم مدیریت‌شده با نشست و حافظهٔ ماندگار مستقر می‌کنید، و به رویدادهای ریپو وصلش می‌کنید.	وقتی درست است که بازبین باید زمینه را در یک بازسازی چندپرامپتی نگه دارد، کاری مثل «این سرویس را برای انحراف `(drift)` انطباق ممیزی کن» را به ده‌ها زیرکار بشکند، یا با عامل‌های دیگر هماهنگ شود. اما ارزیابی، رصدپذیری، هزینه و نوبت‌کاری هنگام خرابی هم مال شماست.

سه پرسش که لایه را تعیین می‌کنند

1. **معیار بازبینی‌تان چقدر ویژه است؟** عمومی ← لایهٔ ۱. ویژهٔ تیم یا ریپو ← لایهٔ ۲ یا ۳.
2. **آیا عامل باید چیزی را میان اجراها به یاد بسپارد؟** نه ← لایهٔ ۱ یا ۲. بله (حافظهٔ کدبیس، زمینهٔ چندپرامپتی) ← لایهٔ ۳.
3. **بدترین حالت چیست اگر اشتباه کند؟** نظری پرحرف ← هر لایه‌ای. پس‌رفتی که ادغام شود یا رازی که نشت کند ← لایهٔ ۳، با سرور سیاست `(Policy Server)` پیش از هر فراخوان ابزار `(tool call)`.

قاعدهٔ انتخاب

پایین‌ترین لایه‌ای را بردارید که آنچه اهمیت دارد را پوشش می‌دهد. بیشتر تیم‌ها لایهٔ خود را همان لحظه‌ای کشف می‌کنند که بازبین مدیریت‌شده چیز خاصی را از دست می‌دهد. نمونهٔ واقعی: تیمی در یک شرکت مالی از لایهٔ ۱ شروع کرد و دید بازبین انطباقش چیزهایی را علامت می‌زند که حسابرسان قبلاً تأیید کرده بودند، در حالی که تنها الگوی مهم — اطلاعات هویتی پوشانده‌نشده در گزارش‌ها — را از دست می‌داد. یک مهارت چهل‌خطیِ وارسی انطباق، وصل‌شده به گردش‌کار خودکار ریپو (لایهٔ ۲)، ظرف یک هفته هشدارهای کاذب را به‌شدت کاهش داد. لایهٔ ۳ هنوز لازم نشده است.

لایهٔ ۳ در مقیاس کامل: درک گرافی کد

بازبین سفارشی از یک عامل ساده که تفاوت‌ها را می‌خواند فراتر می‌رود. روی کدبیسی با ده‌ها میلیون خط، بارکردن کد به‌شکل متن تخت در کانتکست ویندو `(context window)` جا کم می‌آورد، و بازیابی `(retrieval)` استاندارد ساختاری را حذف می‌کند که کد را خوانا می‌سازد: کلاس به فایلی تعلق دارد، و فراخوان تابعی به سند نیازمندی‌ای که ده سال پیش نوشته شده ردیابی می‌شود. الگویی که در بزرگ‌ترین نوسازی‌های کد قدیمی پدیدار شده، ساختن عامل روی یک **گراف دانش** است: کد، مستندات، تیکت‌ها و اسناد طراحی را در پایگاه‌داده‌ای گرافی می‌ریزید و سپس عامل‌ها سه شیوهٔ بازیابی را ترکیب می‌کنند:

- **پیمایش گراف** برای پرسش‌های ساختاری: «هر تابعی که به‌طور متعدی فلان تابع را صدا می‌زند.»
- **جست‌وجوی بُرداری** برای پرسش‌های معنایی: «کدی پیدا کن که کاری را می‌کند که این پاراگراف توصیف می‌کند.»
- **جست‌وجوی تمام‌متن** برای تطبیق دقیق شناسه‌ها.

ترکیب این سه به پرسش «اگر این را عوض کنم چه چیزی خراب می‌شود؟» با نقشهٔ اثر دقیق پاسخ می‌دهد، نه با حدسی مطمئن.

نیمهٔ دوم ماجرا **تجزیه** است. یک عامل که به او بگویید «این ماژول را بازسازی کن» شکست می‌خورد. همان کار را روی خط لولهٔ زیرعاملی تقسیم کنید — عامل جست‌وجو که گراف را می‌کاود، عامل داستان که نیازمندی‌ها را ثبت می‌کند، عامل اثر که عوارض جانبی را پیش‌بینی می‌کند، عامل تجزیهٔ کار که واحدهای اتمی می‌سازد، و تازه بعد عامل کدنویسی — و کار مدیریت‌پذیر می‌شود. در کدبیس‌های میلیون‌خطی گزارش شده که کار بازسازی معادل از دو هفته به چند ساعت رسیده است.

۱۰. پایداری و خستگی تأیید

پدیده‌ای تازه

پژوهش‌ها نشان می‌دهند کاربران پرکار هوش مصنوعی به‌مراتب بیشتر از غیرکاربران در معرض فرسودگی شدید قرار دارند. زیر جریان مداوم تأییدهای خرد — بهبود یک خط، تنظیم یک فراخوان ابزار — توسعه‌دهنده به‌صورت بی‌فکر و از روی عادت روی «تأیید» کلیک می‌کند. این شکلی از خستگی کم‌شدت است که در آن تیم فقط برای همپا ماندن با سرعت ماشین، از وارسی کارش دست می‌کشد — و شما ریسک می‌کنید که توسعه‌دهندگان توجه به جزئیات را از دست بدهند.

برای محافظت از تیم، از نظارت مداوم به مرزهای ساختارمند بروید:

- **ساعت‌های آرام دیجیتال:** مرزهایی بگذارید تا درخواست‌های تأیید به عصرها و آخر هفته‌ها نشت نکنند.
- **جلسه‌های بینش عامل:** جلسه‌های هفتگی برگزار کنید که در آن‌ها توسعه‌دهندگان الگوهای شناسایی‌شده به‌دست همتای هوشمندشان را به اشتراک می‌گذارند، و کشف‌های فردی به دانش سازمانی مشترک بدل می‌شود.

۱۱. توسعهٔ اعتماد صفر: ساختن شبکهٔ ایمنی

داستان واقعی پنجاه ایمیل

هنگام یک به‌روزرسانی معمولی کد، توان و مرزهای مرورگر داخلی یک محیط عامل‌محور آشکار شد. این قابلیت به عامل اجازه می‌دهد بدون نیاز به اعتبارنامهٔ ورود با برنامه‌های در حال توسعه تعامل کند و برای تست تجربهٔ کاربری بی‌نظیر است. اما در **حالت تأیید خودکار**، می‌تواند سریع‌تر از فکر انسان عمل کند.

پرامپت ساده‌ای برای ساختن یک دکمه، زنجیرهٔ واکنش غیرمنتظره‌ای را راه انداخت: عامل مرورگر خودگردان روی دکمهٔ تازه کلیک کرد که برای عامل ایمیل در نظر گرفته شده بود. چون نشانی مشخصی تعریف نشده بود، عامل توهم کرد و به عاملی قدیمی و منسوخ بدون هیچ گاردریل (`guardrail`) ایمیلی وصل شد. نتیجه؟ **پنجاه همکار ایمیل‌های جعلی پر از محتوای توهمی دریافت کردند.**

درس این حادثه

این رویداد **مخاطرهٔ توهم زمینه** (Context hallucination) را برجسته کرد: وقتی هوش مصنوعی دادهٔ کافی ندارد، گاهی شکاف‌ها را با هر رشته‌ای که در زمینه‌اش موجود است پر می‌کند، از جمله اطلاعات حساسی مثل نشانی‌های ایمیل سخت‌کدشده یا نشانی‌های خصوصی.

ممکن است اگر فقط یک ایمیل باشد جزئی به‌نظر برسد. اما به آنچه عامل انجام می‌داد فکر کنید: داشت دستورش را با داده‌ای که در اختیار داشت انجام می‌داد، **بدون هیچ وارسی‌ای بر اینکه آیا باید این کار را بکند یا نه**. این هستهٔ مخاطره در سامانه‌های خودگردان است. گاردریل‌ها اختیاری نیستند؛ همان چیزی‌اند که نمی‌گذارند ابزاری مفید به ابزاری غیرقابل‌پیش‌بینی بدل شود.

پارادوکس گاردریل

با هل‌دادن مرزهای هوش مصنوعی عامل‌محور به پارادوکسی برمی‌خوریم: عامل‌ها باید آن‌قدر خودگردان باشند که مسائل پیچیده را حل کنند، اما نمی‌توان ریسک یاغی‌شدنشان را در محیط سازمانی پذیرفت. عاملی را تصور کنید که مأمور «حل اختلاف‌های مشتری» است: برای مؤثر بودن به دادهٔ مشتری، ابزار ایمیل و سامانه‌های داخلی نیاز دارد. اما چالش این است که مطمئن شویم ناخواسته کل پایگاه داده را ایمیل نمی‌کند یا کد اختصاصی را به اشتراک نمی‌گذارد.

چرا سخت‌کدکردن قید در سیستم پرامپت شکننده است

عامل‌های خودگردان را مدل‌هایی می‌رانند که **احتمالاتی‌اند، نه قطعی** (deterministic). زمینه‌ها سرریز می‌کنند و می‌شود عامل را با تزریق پرامپت «قانع» کرد که قواعد را دور بزند. برای ساخت سکوهای سطح تولید، به حکمرانی **بیرونی و دست‌نیافتنی** نیاز است.

پنج لایهٔ شبکهٔ ایمنی

۱. سندباکس

فراتر از پاک‌سازی (sanitization) رشته‌ها، امنیت واقعی محیط اجرای محدود می‌خواهد. حتی با پالایش دقیق خروجی، مدل ممکن است کدی تولید کند که از نظر نحوی معتبر اما از نظر منطقی مخرب است. با اجرای کارهای عامل‌محور درون کانتینرهای زودگذر (ephemeral) و کم‌امتیاز که از شبکهٔ اصلی و فایل‌سیستم حساس ایزوله‌اند، **شعاع انفجاری** می‌سازید که از زیرساخت اصلی محافظت می‌کند. در این مدل، اگر عامل فریب بخورد و فرمانی مخرب اجرا کند، آسیب به نمونه‌ای دورریختنی محدود می‌ماند که می‌توان بی‌عواقب پاکش کرد.

در برخی محیط‌ها این شبکهٔ ایمنی با یک کلید در تنظیمات فعال می‌شود. اگر سندباکسی قابل‌حمل و ابری برای تیم لازم دارید، فضای کاری عامل را در کانتینر (container) بگذارید: فایل کانتینر سفارشی بنویسید، اعتبارنامهٔ ابری محدود تزریق کنید، و ابزار را وادار کنید کاملاً درون کانتینر اجرا شود. در هر دو مدل، اگر عامل فریب بخورد، به خطای مجوز سختی در سطح هسته برمی‌خورد و سامانهٔ میزبان کاملاً دست‌نخورده می‌ماند.

۲. انسان در حلقه

هرچند خودکارسازی هدف است، عملیات پرمخاطره به پروتکل انسان-در-حلقه به‌عنوان ایمنی نهایی نیاز دارند. این یعنی پیاده‌سازی دروازه‌های (gate) «وارسی» برای کنش‌هایی که نمای مخاطرهٔ مشخصی دارند: انتشار کد در تولید، تغییر اسکیمای پایگاه داده، یا آغاز تراکنش مالی. با ارائهٔ قصد پاک‌شدهٔ عامل به ناظر انسانی برای تأیید دستی، سرعت هوش مصنوعی با قضاوت سنجیدهٔ توسعه‌دهنده متوازن می‌شود. این تضمین می‌کند که در حالی که عامل کار سنگین را انجام می‌دهد، مسئولیت نهایی یکپارچگی معماری محکم در دست انسان می‌ماند.

۳. پوشش تست تولیدشده با هوش مصنوعی

جهش کد تولیدشده، جابه‌جایی از تست دستی به پوشش تست تولیدشده با هوش مصنوعی را ضروری می‌کند. در محیطی پرسرعت، «توسعهٔ تست‌محور» با گماشتن ماشین به نوشتن همان تست‌هایی که خروجی خودش را اعتبارسنجی می‌کنند به واقعیت بدل می‌شود. این فرایند یعنی وادارکردن عامل به تولید یک تست واحدِ شکست‌خورده یا فرمان بازتولید **پیش از** هر تلاشی برای اصلاح. با جاسازی این تست‌ها در کدبیس، هر تکرار سریع با مجموعه‌ای از وارسی‌های راستی‌آزمایی‌پذیر پشتیبانی می‌شود.

۴. ارزیابی

چرا برای کدی که از مدل‌ها استفاده می‌کند به وارسی کیفیت ویژه نیاز داریم؟ تست‌های سنتی برای سامانه‌هایی که خروجی‌شان **تولید** می‌شود و نه **محاسبه**، ناکافی‌اند. عامل — یا هر جزء مدل‌محور دیگری — می‌تواند صد تست واحد را روی ابزارهایش پاس کند و همچنان به‌شکل تماشایی شکست بخورد: با انتخاب ابزار اشتباه، بازنویسی بد یک پاسخ حیاتی، یا توهم یک واقعیت.

> **تست در برابر ارزیابی** (Tests vs. Evals)
>
> تست واحد می‌پرسد «آیا تابع مقدار درست را برگرداند؟» — پاسخی صفر و یکی. ارزیابی می‌پرسد «آیا رفتار عامل دست‌کم به‌خوبی مبناست؟» — نمره‌ای از داور (judge) مدل‌زبانی، وارسی مسیری که تنوع ترتیب فراخوان ابزار را تحمل می‌کند، و دروازه‌ای که وقتی کیفیت زیر حاشیه‌ای قابل‌پیکربندی بیفتد فعال می‌شود، نه وقتی ادعایی نقض شود.
>
> **تست‌ها پس‌رفت** (regression) **قطعی را می‌گیرند، اما ارزیابی انحراف رفتاری را می‌گیرد.** و حاشیهٔ خطا نقصی نیست که باید حذف شود؛ ویژگی ذاتی مدل است و راهبرد تست باید با آن کنار بیاید.

۵. سرور سیاست

سرور سیاستِ ترکیبی کنش‌ها را پیش از رسیدن به سامانه‌های بیرونی رهگیری (interception) می‌کند و در دو لایه کار می‌کند:

- **دروازهٔ ساختاری** (Structural gating) **(چراغ راهنما):** قواعد قطعی بر پایهٔ نقش و محیط. وارسی‌هایی سریع و صفر و یکی — مثلاً نقش «بیننده» نمی‌تواند از ابزار ارسال ایمیل استفاده کند. این لایه بدون پرسیدن از هیچ مدلی، از نقض معماری جلوگیری می‌کند.
- **دروازهٔ معنایی** (Semantic gating) **(داور هوشمند):** مدلی ثانویه و تخصصی که قصد و محتوای کنش پیشنهادی را در برابر راهنماهای حریم خصوصی به زبان طبیعی بازرسی می‌کند. این برای وقتی است که **ابزار مجاز است اما نحوهٔ استفاده سیاست** (policy) **را نقض می‌کند:** مدیر می‌تواند ایمیل بفرستد، اما نباید اطلاعات هویتی پوشانده‌نشده بفرستد. همین‌جاست که قواعد ساختاری کم می‌آورند، چون نمی‌توان هر نشت ممکن را با عبارت باقاعده گرفت.

```
environments:
  localhost:
    blocked_tools:
      - send_email

roles:
  viewer:
    allowed_tools:
      - list_files
      - read_file
```

جریان اجرا هنگام تصمیم عامل به استفاده از ابزار رهگیری می‌شود:

1. **وارسی ساختاری:** آیا این ابزار برای این نقش و محیط مجاز است؟
2. **وارسی معنایی:** آیا آرگومان‌ها امن‌اند؟ (برای نمونه، نشانی ایمیل پوشانده‌نشده نقض محسوب می‌شود.)
3. **اجرا:** اگر هر دو پاس شدند، ابزار اجرا می‌شود. وگرنه پیام «نقض سیاست» به عامل برمی‌گردد تا خودش را اصلاح کند یا به‌شکلی کنترل‌شده متوقف شود.

این کار شبکهٔ ایمنی‌ای می‌سازد که **منطق اجرا را از منطق حکمرانی جدا می‌کند** — تفکیک مسئولیتی حیاتی برای نرم‌افزار سازمانی.

بهداشت زمینه و پاک‌سازی پرامپت

برای مهار (harness) مخاطرهٔ توهم زمینه، بهداشت زمینه (Context hygiene) را با میان‌افزاری پیاده کنید که اطلاعات هویتی را می‌پوشاند و جانشین تزریق می‌کند. با جایگزین‌کردن اطلاعات شناسایی‌پذیر با جانشین‌های عمومی در قالب‌ها، مطمئن می‌شوید عامل روی **دادهٔ استریل‌شده** کار می‌کند. افزون بر این، همهٔ خروجی‌های عامل باید پاک‌سازی شوند تا از تزریق پرامپت و تعامل‌های یاغی با رابط کاربری جلوگیری شود.

```
def resolve_context(template_str, override_state=None):
    """Replace [[VARIABLE_NAME]] with runtime state or env values."""
    state = override_state or {}

    def replacement(match):
        var = match.group(1).strip()
        if state.get(var) is not None:       # 1. runtime override
            return str(state[var])
        if os.environ.get(var) is not None:  # 2. validated env var
            return os.environ[var]
        return match.group(0)                # 3. leave unresolved: no silent fa

    return re.sub(r'\[\[([^\]]+)\]\]', replacement, template_str)
```

این ابزار باید مستقیم در خط لولهٔ اجرای عامل به‌عنوان گام اعتبارسنجی قرار داده شود: فراخوان‌های ورودی ابزار را پیش از اجرا رهگیری کنید تا همهٔ رشته‌های تزریق‌شده پویا استریل شوند. با اعمال این مرز، هر تلاش عامل برای اجرای کنشی — مثل فرستادن ایمیل یا پرس‌وجوی یک منبع ابری — رهگیری می‌شود و موتور، جانشین‌های عمومی را امن و بی‌صدا به دارایی‌های تستیِ مجاز ترجمه می‌کند. نتیجه: نیاز به سخت‌کدکردن اطلاعات حساس در مجموعه‌تست‌ها یا سیستم پرامپت‌ها کاملاً از بین می‌رود.

۱۲. جمع‌بندی دوره

جلسه	پرسش محوری	پاسخی که ساختیم
۱	چه چیزی واقعاً عوض شده؟	گذار از نحو به قصد؛ طیف کدنویسی شهودی تا مهندسی عامل‌محور؛ و اینکه **عامل = مدل + مهار**.
۲	عامل چطور با جهان حرف می‌زند؟	پروتکل‌های باز به‌جای پوشش‌های سفارشی: ابزار، همکاری عامل با عامل، رابط کاربری مولد، و تجارت خودگردان.
۳	عامل چطور می‌داند «چگونه»؟	مهارت به‌عنوان نخستین حافظهٔ رویه‌ای (Procedural memory) باورپذیر، با افشای تدریجی (Progressive disclosure) و ارزیابی سخت‌گیرانه.
۴	چه چیزی خراب می‌شود و چطور می‌سنجیم؟	هفت ستون امنیت، و هفت بُعد ارزیابی — دو محور مستقل اعتماد.
۵	چطور این‌ها را به تولید می‌رسانیم؟	مشخصات به‌عنوان منبع حقیقت، فرایند تیمی بازطراحی‌شده، و شبکهٔ ایمنی اعتماد صفر.

در کمتر از یک سال، چرخهٔ توسعهٔ نرم‌افزار به‌شکل چشمگیری سریع‌تر شده است. اما آن سرعت جابه‌جایی مهمی را آشکار کرد: هوش مصنوعی گلوگاه **تولید کد** را حذف کرده و قید را به پایین‌دست برده، به انسان‌هایی که باید این خروجی را بازبینی، تست و یکپارچه کنند. این یک **بار شناختی مشترک** است: انسان‌ها معمارانی‌اند که مشخصات تست، مشخصات یکپارچگی و نقشه‌های عملیاتی را می‌نویسند، در حالی که هوش مصنوعی کار سنگین نوشتن کد تست، انجام یکپارچه‌سازی و مدیریت جزئیات ریز عملیاتی را بر عهده می‌گیرد.

> پرامپت‌های بهتر و مدل‌های سریع‌تر به‌تنهایی گلوگاه یکپارچگی را حل نمی‌کنند. موفقیت به تحول پویایی تیم، پالایش همکاری با عامل‌ها، و تعیین مرزهای سخت‌گیرانه برای ابزارهایی بستگی دارد که هرگز نمی‌خوابند. چالش از تولید صرف کد، به هماهنگ‌کردن سامانه‌هایی منتقل شده که کار را راستی‌آزمایی، یکپارچه و مستقر می‌کنند.

۱۳. خودآزمایی

1. «توهم سرعت» را در یک جمله توضیح دهید و بگویید در تیم شما چطور بروز می‌کند.
2. اگر مشخصات پروژهٔ شما فردا از بین برود، چقدر از کدبیس‌تان قابل بازتولید است؟ این عدد چه چیزی دربارهٔ کیفیت مشخصات شما می‌گوید؟
3. یک نیازمندی مبهم از پروژهٔ خودتان را بردارید و به سه سناریوی «با فرض / وقتی / آنگاه» تبدیل کنید.
4. در پروژهٔ شما، چه چیزی باید در پوشهٔ مشخصات باشد، چه چیزی در مهارت‌ها، و چه چیزی در سیستم پرامپت؟
5. آخرین باری که با عامل اشکالی را رفع کردید، پرامپت‌تان علامت‌محور بود یا شاهدمحور؟
6. کدام‌یک از سه دستهٔ خطای فرایندی (تعارض ادغام، گره‌خوردگی بازبینی، پاره‌پاره‌شدن زمینه) بیشترین هزینه را به تیم شما تحمیل می‌کند؟
7. تیم شما به کدام لایهٔ بازبین خودکار نیاز دارد؟ سه پرسش تصمیم‌گیری را برای خودتان پاسخ دهید.
8. آیا در تیم شما نشانه‌های خستگی تأیید `(Approval fatigue)` دیده می‌شود؟ چه مرزی می‌توانید همین هفته بگذارید؟
9. داستان پنجاه ایمیل را بازخوانی کنید: کدام‌یک از پنج لایهٔ شبکهٔ ایمنی می‌توانست جلویش را بگیرد، و در کدام نقطه؟
10. یک نمونه بیاورید که در آن ابزار مجاز است اما نحوهٔ استفاده باید مسدود شود. این قاعده را به زبان طبیعی بنویسید.

منبع: این جزوه خلاصه و بازنویسی آموزشی مقالهٔ `Spec-Driven Production Grade Development in the Age of Vibe Coding` نوشتهٔ لی بونسترا است (مجموعهٔ مقالات عامل‌ها، می ۲۰۲۶). ترجمهٔ کامل فارسی در همین پوشه در دسترس است.

واژه‌نامهٔ این جلسه

اصطلاحاتی که در این سند به کار رفته‌اند، به‌همراه معادل انگلیسی‌شان. فهرست کامل 159 اصطلاح دوره در واژه‌نامهٔ دوره آمده است.

اصطلاح	معادل انگلیسی	توضیح
ابزار	`tool`	تابعی که عامل می‌تواند فرا بخواند تا در جهان بیرون کنش کند.
ارزیابی	`evaluation`	سنجش رفتار غیرقطعی با نمره و باند تحمل، در برابر ادعای صفر و یکی تست.
افشای تدریجی	`Progressive disclosure`	بارگذاری سه‌سطحی: فراداده همیشه، بدنه هنگام فعال‌شدن، منابع همراه فقط در صورت نیاز.
انحراف	`drift`	دورشدن تدریجی رفتار از آنچه انتظار می‌رفت.
بازیابی	`retrieval`	آوردن دانش بیرونی به زمینه در لحظهٔ نیاز.
بهداشت زمینه	`Context hygiene`	پوشاندن اطلاعات هویتی و تزریق جانشین، تا عامل روی دادهٔ استریل‌شده کار کند.
تأخیر	`latency`	زمان سپری‌شده تا دریافت پاسخ.
تأیید مشروط	`Conditional LGTM`	تأیید بازبین مشروط به سبزشدن همهٔ تست‌های خودکار؛ سپس ادغام خودکار.
تجزیه	`decomposition`	شکستن کاری بزرگ به زیرکارهای اتمی.
تست	`test`	وارسی قطعی رفتار کد. «یونیت تست» = آزمون واحد.
تست در برابر ارزیابی	`Tests vs. Evals`	تست بخش قطعی را می‌سنجد؛ ارزیابی بخش غیرقطعی را: مسیر طی‌شده، انتخاب ابزار و کیفیت پاسخ نهایی.
توسعهٔ رفتارمحور	`BDD`	توصیف رفتار سامانه از دید کاربر، پیش از نوشتن کد، با زبان طبیعیِ ساختارمند.
توهم	`hallucination`	تولید بااطمینانِ محتوای نادرست به‌دست مدل.

اصطلاح	معادل انگلیسی	توضیح
توهم زمینه	`Context hallucination`	وقتی عامل داده ندارد، شکاف را با هر رشته‌ای که در زمینه‌اش هست پر می‌کند — از جمله اطلاعات حساس.
توهم سرعت	`Illusion of Speed`	تولید سریع کد، سرعت انتشار نیست؛ گلوگاه فقط جابه‌جا شده است.
توکن	`token`	واحد کوچکی از متن که مدل پردازش می‌کند.
حافظهٔ رویه‌ای	`Procedural memory`	به‌خاطرسپردن «چگونه انجام دادن»؛ در برابر حافظهٔ رویدادی (چه گذشت) و معنایی (چه می‌دانیم).
حالت بی‌محابا	`YOLO mode`	حالت تأیید خودکار؛ عامل سریع‌تر از فکر انسان عمل می‌کند.
حلقه	`loop`	چرخهٔ ادراک ← برنامه ← کنش ← مشاهده که عامل در آن می‌چرخد.
خستگی تأیید	`Approval fatigue`	کلیک بی‌فکر روی «تأیید» زیر سیل تأییدهای خرد؛ شکلی از فرسودگی کم‌شدت.
خودگردان	`autonomous`	توان اجرای کار بدون تأیید انسانی در هر گام.
داور	`judge`	مدلی که خروجی مدل دیگر را در برابر سنجه نمره می‌دهد.
دروازه	`gate`	نقطهٔ وارسی که اجرا را تا برآورده‌شدن شرطی متوقف می‌کند.
دروازهٔ ساختاری	`Structural gating`	قواعد قطعی بر پایهٔ نقش و محیط؛ وارسی‌های سریع و صفر و یکی.
دروازهٔ معنایی	`Semantic gating`	مدلی تخصصی که قصد و محتوای کنش پیشنهادی را در برابر سیاست‌های زبان طبیعی می‌سنجد.

اصطلاح	معادل انگلیسی	توضیح
دستور	`instruction`	قاعده‌ای که رفتار عامل را شکل می‌دهد، در برابر داده‌ای که پردازش می‌کند.
دیباگ	`debug`	یافتن و رفع علت ریشه‌ای خطا.
دیپلوی	`deploy`	انتشار کد روی محیط اجرا.
رانتایم	`runtime`	محیطی که عامل یا برنامه در آن اجرا می‌شود.
رهگیری	`interception`	گرفتن یک کنش پیش از رسیدنش به سامانهٔ بیرونی.
ریپو	`repository`	مخزن کد تحت کنترل نسخه.
زمینه	`context`	اطلاعاتی که عامل در لحظه در اختیار دارد. پایهٔ «مهندسی زمینه» و «پوسیدگی زمینه».
زمینهٔ ایستا / پویا	`Static`	ایستا همیشه بارگذاری می‌شود (هویت عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی سند).
زودگذر	`ephemeral`	کوتاه‌عمر و دورریختنی؛ پس از کار بازنشانی می‌شود.
زیرعامل	`subagent`	عاملی که عامل دیگری برای کاری مشخص فرا می‌خواند.
ستارهٔ شمالی معماری	`Architectural North Star`	نقش مشخصات خوب: جلوگیری از پاره‌پاره‌شدن زمینه و بازی «تلفن».
سرور سیاست	`Policy Server`	حکمرانی بیرونی و دست‌نیافتنی که کنش‌ها را پیش از رسیدن به سامانه‌های بیرونی رهگیری می‌کند.
سندباکس	`sandbox`	محیط اجرای ایزوله و کم‌امتیاز برای مهار شعاع انفجاری.

اصطلاح	معادل انگلیسی	توضیح
سیاست	`policy`	قاعدهٔ حکمرانی که بیرون از مدل تعریف و اعمال می‌شود.
سی‌آی / سی‌دی	`CI`	یکپارچه‌سازی و استقرار پیوسته.
عامل	`agent`	مدل به‌علاوهٔ مهار؛ توان استدلال، به‌کارگیری ابزار و اجرای کار.
عامل یاغی	`Rogue agent`	وقتی به مدل «حس‌وحال» بدهی به‌جای «نقشه»، حدس می‌زند — و در نرم‌افزار سازمانی، حدس یعنی حادثه.
فراخوان ابزار	`tool call`	یک نوبت مشخص از به‌کارگیری ابزار، با آرگومان‌ها و نتیجه‌اش.
قصد / نیت	`intent`	خواستهٔ واقعی کاربر، در برابر آنچه لفظاً گفته. ریشهٔ «انحراف قصد» و «رضایت قصد».
قطعی / غیرقطعی	`deterministic`	قطعی یعنی ورودی یکسان همیشه خروجی یکسان می‌دهد؛ مدل‌ها غیرقطعی‌اند.
قید	`constraint`	محدودیت سختی که فضای کنش عامل را کوچک می‌کند.
هزینهٔ قالب	`Format tax`	هزینهٔ استدلالی که قالب نامناسب ورودی تحمیل می‌کند.
مسیر	`trajectory`	توالی گام‌های استدلال و فراخوان ابزار که عامل تا رسیدن به پاسخ طی می‌کند.
مشخصات	`spec`	سند قصد و رفتار مورد انتظار؛ منبع حقیقت برای عامل و انسان.
مهار	`harness`	هرچه پیرامون مدل قرار می‌گیرد: پرامپت، ابزار، سندباکس، ارکستراسیون، قلاب. عامل = مدل + مهار.

اصطلاح	معادل انگلیسی	توضیح
مهارت	`skill`	پوشهٔ دانش رویه‌ای که عامل فقط هنگام نیاز بارگذاری می‌کند.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها با نظارت انسانی بر معماری.
نشت	`leak`	خروج ناخواستهٔ اطلاعات حساس از مرز سامانه.
نشست	`session`	کل یک گفت‌وگوی چندنوبتی، از آغاز تا پایان کار.
هویت	`identity`	اینکه چه کسی یا چه عاملی دارد کنش می‌کند.
پاره‌پاره‌شدن زمینه	`Context fragmentation`	وقتی عامل به عکس‌های قدیمی فایل‌ها نگاه می‌کند و رشتهٔ ماجرا را گم می‌کند.
پاک‌سازی	`sanitization`	بی‌اثرکردن ورودی یا خروجی خطرناک پیش از استفاده.
پرامپت	`prompt`	متنی که به مدل داده می‌شود. پیش‌تر «درخواست» بود که با request اشتباه می‌شد.
پس‌رفت	`regression`	خراب‌شدن چیزی که پیش‌تر کار می‌کرد.
پوشش	`coverage`	سهمی از رفتار که آزمون یا ارزیابی واقعاً می‌سنجد.
پول‌ریکوئست	`pull request`	درخواست ادغام تغییرها در شاخهٔ اصلی.
کانتکست ویندو	`context window`	بیشینهٔ توکنی که مدل در یک نوبت می‌تواند ببیند.
کانتینر	`container`	واحد بسته‌بندی و اجرای ایزولهٔ نرم‌افزار.

اصطلاح	معادل انگلیسی	توضیح
کد دورریختنی	`Disposable code`	وقتی مشخصات محکم باشد، کل کدبیس بارها بازتولید می‌شود؛ پس دلبستگی عاطفی به کد معنا ندارد.
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
گاردریل	`guardrail`	قید بیرونی که کنش‌های مجاز عامل را محدود می‌کند.
گردش‌کار	`workflow`	توالی گام‌های یک کار تکرارشونده.
گره‌خوردگی بازبینی	`Review gridlock`	پول‌ریکوئست غول‌آسا که به زنجیره‌ای از پول‌ریکوئست‌های وابسته تبدیل می‌شود.
